Aprendizaje de Máquina
Sesión 26: Ensambles y bosques aleatorios

Angel Rodríguez

Avisos:

- Último examen parcial: martes 10 de Noviembre. Guía la subo hoy en la noche.
- Exposición proyectos finales:
  1. jueves 26 de Noviembre en horario y salón de clase (4 equipos).
  2. jueves 3 de Diciembre, de 19:00 a 20:59 horas en el CN218 (4 equipos).
- Fecha de entrega proyecto final: miércoles 25 de Noviembre a las 23:59.

Proyecto final: análisis y clasificación de textos

Objetivo

Desarrollar un sistema que **descubra grupos temáticos** en documentos y **clasifique textos nuevos**, justificando las decisiones mediante experimentos reproducibles.

Componentes del proyecto

1. **Representación (20 %):** limpieza, BoW y TF–IDF.
2. **No supervisado (25 %):** K-means, jerárquico, métricas y reducción de dimensionalidad.
3. **Supervisado (25 %):** comparar 3 clasificadores y seleccionar el mejor.
4. **Generalización (20 %):** sobreajuste, regularización, dimensionalidad y curvas de aprendizaje.
5. **Discusión (10 %):** interpretación, limitaciones y conclusiones.

Entregables

- Reporte escrito.
- Código Python reproducible.
- Presentación (12 min) y preguntas (8 min).

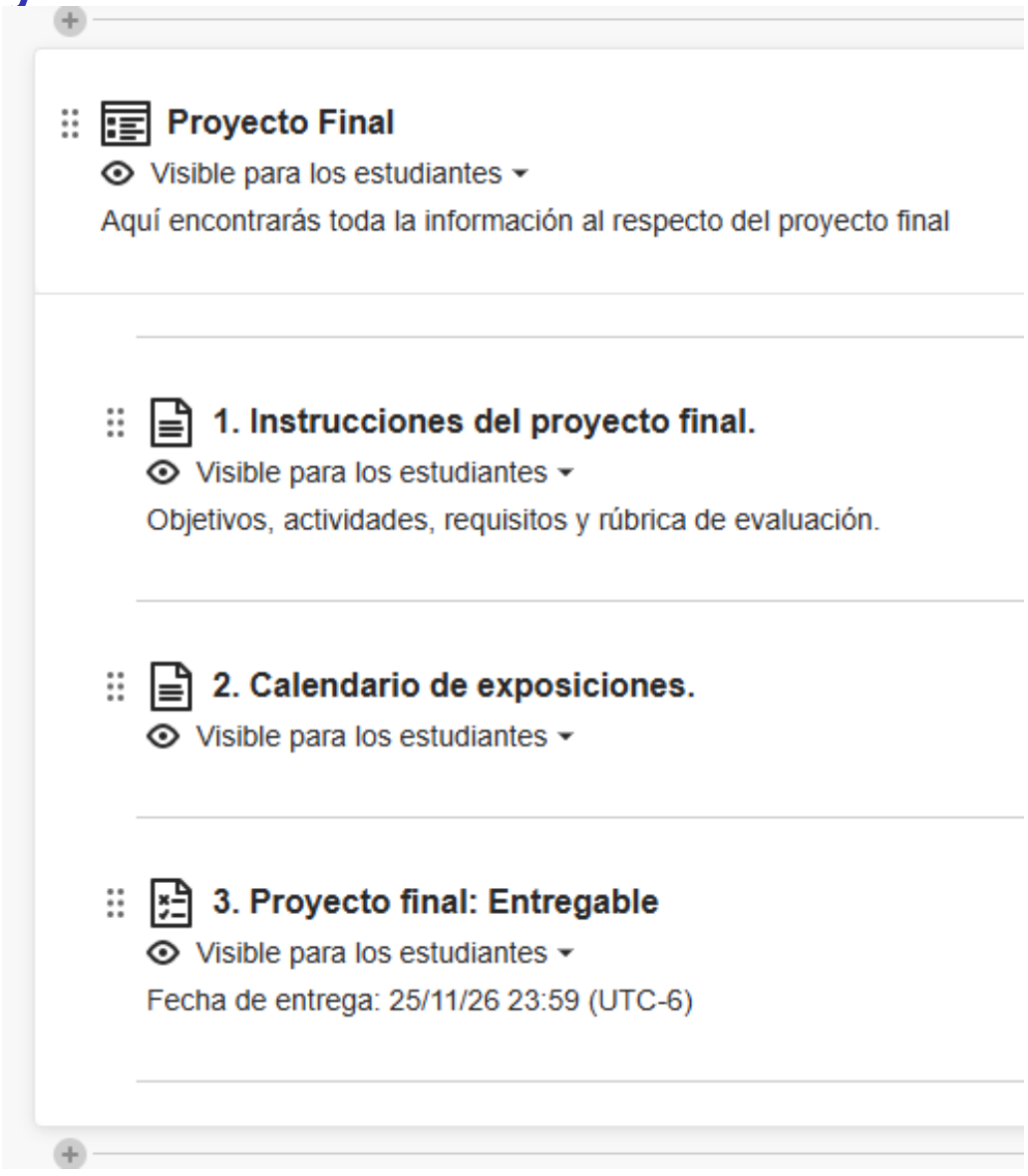
Condiciones mínimas

≥3,000 textos, ≥4 categorías, 2 representaciones, 2 agrupamientos, 3 clasificadores y 4 experimentos.

Proyecto final: dinámica general

- El proyecto se realizará en **equipos de dos personas**.
- **Entregables:** cada equipo deberá entregar:
  - Un **reporte en PDF**.
  - El **código Python** en un archivo aparte y documentado.
- **Evaluación:**
  - Reporte en PDF: **30 %** (calificación por equipo).
  - Exposición oral: **70 %** (calificación individual).
- Cada equipo realizará una **exposición oral de 12 minutos** para presentar su proyecto.
- **Participación individual:** se harán **preguntas a cada integrante** para verificar que todos comprendan el proyecto completo.
- **Importante:** se **restarán puntos por respuestas equivocadas**. Todos deben dominar la teoría, el código y los resultados.

Proyecto final: blackboard



Ensemble Learning: idea general

Aprendizaje por ensambles

Un **ensamble** combina las predicciones de varios modelos para construir un predictor que, frecuentemente, generaliza mejor que sus componentes individuales.

La idea está relacionada con la *sabiduría colectiva*: la combinación de distintas opiniones puede producir una respuesta más precisa que una opinión individual.

- Los modelos individuales se denominan **predictores base**.
- Pueden utilizar diferentes algoritmos o diferentes subconjuntos de entrenamiento.
- Sus predicciones se combinan mediante votación, promedios o un modelo adicional.

Idea fundamental

Un ensamble funciona especialmente bien cuando sus predictores son suficientemente precisos y cometen errores diferentes.

Principales métodos de ensamble

1. **Voting:** combina las predicciones de distintos clasificadores mediante votación.
2. **Bagging y Pasting:** entrena múltiples modelos utilizando diferentes subconjuntos de datos.
3. **Random Forests:** combina árboles de decisión incorporando aleatoriedad en los datos y las características.
4. **Boosting:** construye modelos secuencialmente, intentando corregir los errores anteriores.
5. **Stacking:** utiliza un modelo adicional para aprender cómo combinar las predicciones.

Objetivo

Mejorar la capacidad de generalización, frecuentemente reduciendo la varianza o el sesgo del predictor final.

Voting Classifiers: votación mayoritaria

Supongamos que entrenamos M clasificadores $f_1, \ldots, f_M$ sobre un problema de clasificación.
Cada clasificador produce una predicción $\hat{y}_j(x)$ para una observación x.

Hard Voting

La predicción del ensamble es la clase que recibe más votos:

$$\hat{y}(x) = \arg\max_k \sum_{j=1}^{M} \mathbb{I}\big(\hat{y}_j(x) = k\big)$$

donde $\mathbb{I}(\cdot)$ es la función indicadora.

Por ejemplo, si tres clasificadores predicen

$$\hat{y}_1(x) = 1, \qquad \hat{y}_2(x) = 1, \qquad \hat{y}_3(x) = 0,$$

entonces el ensamble predice $\hat{y}(x) = 1$.

Soft Voting: votación probabilística

En lugar de votar directamente por una clase, cada modelo estima las probabilidades de pertenencia a cada clase.

Soft Voting

Para M clasificadores, calculamos

$$\overline{P}(y = k \mid x) = \frac{1}{M} \sum_{j=1}^{M} P_j(y = k \mid x).$$

La predicción final es

$$\boxed{\hat{y}(x) = \arg\max_k \overline{P}(y = k \mid x).}$$

- **Hard voting:** cada modelo aporta una clase.
- **Soft voting:** cada modelo aporta probabilidades.
- Soft voting considera la confianza de los modelos.

Bagging y Pasting

Otra forma de generar diversidad consiste en entrenar el mismo algoritmo sobre diferentes muestras aleatorias.

Bagging: Bootstrap Aggregating

Cada predictor se entrena con una muestra aleatoria **con reemplazo** del conjunto de entrenamiento.
Una observación puede aparecer varias veces en la muestra de un mismo predictor.

Pasting

Cada predictor se entrena con una muestra aleatoria **sin reemplazo**.
Una observación aparece como máximo una vez en la muestra de cada predictor.

En ambos casos, una observación puede ser utilizada para entrenar distintos predictores.

Agregación de modelos en Bagging

Supongamos que entrenamos M modelos $f_1, \ldots, f_M$ sobre diferentes muestras.

Regresión

La predicción suele calcularse mediante el promedio:

$$\hat{f}(x) = \frac{1}{M} \sum_{j=1}^{M} f_j(x).$$

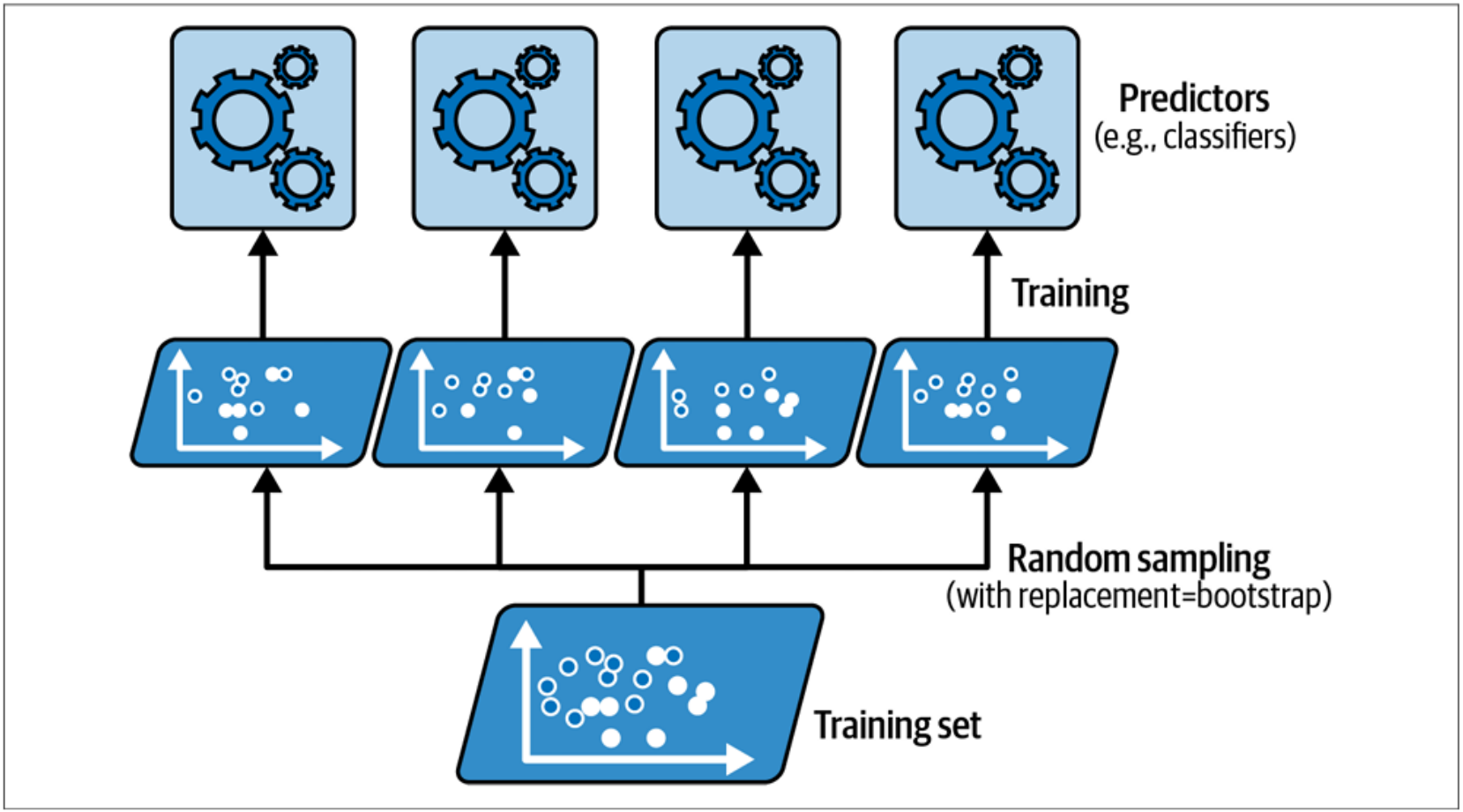
Clasificación

Una opción es elegir la clase más frecuente:

$$\hat{y}(x) = \text{mode}\Big(\hat{y}_1(x), \ldots, \hat{y}_M(x)\Big).$$

La agregación permite reducir la variabilidad de las predicciones individuales. En particular, bagging suele ser efectivo con modelos de **bajo sesgo y alta varianza**, como los árboles de decisión profundos.

Bagging y pasting escalan bien:



¡Los clasificadores pueden entrenarse en paralelo!

Out-of-Bag Evaluation

En bagging, cada predictor recibe una muestra bootstrap del conjunto de entrenamiento.
Al utilizar m extracciones con reemplazo, la probabilidad de que una observación no sea seleccionada es

$$P(\text{no seleccionada}) = \left(1 - \frac{1}{m}\right)^m.$$

Cuando m es grande,[1]

$$\left(1 - \frac{1}{m}\right)^m \longrightarrow e^{-1} \approx 0{,}368.$$

Consecuencia: En promedio, aproximadamente el 63,2 % de las observaciones distintas aparecen en cada muestra bootstrap. El 36,8 % restante queda **fuera de la muestra** (*out-of-bag*, OOB).
Estos datos pueden utilizarse para evaluar los modelos sin reservar un conjunto de validación adicional.

[1] $\lim_{n\to\infty}\left(1 + \frac{a}{n}\right)^n = e^a$.

Evaluación Out-of-Bag en Scikit-Learn

```
bag_clf = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=500,
    oob_score=True,
    n_jobs=-1,
    random_state=42
)

bag_clf.fit(X_train, y_train)

print("OOB:", bag_clf.oob_score_)
print("Test:", bag_clf.score(X_test, y_test))
```

Resultados del ejemplo:

$$\text{OOB Accuracy} = 0{,}896, \qquad \text{Test Accuracy} = 0{,}920.$$

- Cada observación es evaluada mediante los modelos que no la utilizaron durante su entrenamiento.
- `oob_score_` contiene la evaluación OOB.

Random Patches y Random Subspaces

El muestreo no tiene que limitarse a las observaciones: también podemos seleccionar características aleatoriamente.

Random Patches

Cada predictor se entrena con:

- Un subconjunto aleatorio de observaciones.
- Un subconjunto aleatorio de características.

Random Subspaces

Se utilizan todas las observaciones de entrenamiento, pero solamente un subconjunto aleatorio de características.

- `max_features`: cantidad de características.
- `bootstrap_features`: permite seleccionar características con reemplazo.

Compromiso

El muestreo de características incrementa la diversidad y puede reducir la varianza, pero también aumentar el sesgo.

Random Forests: bosques aleatorios

Un **Random Forest** es un ensamble de árboles de decisión que incorpora aleatoriedad durante el entrenamiento.

1. Se generan diferentes muestras del conjunto de entrenamiento, normalmente mediante bootstrap.
2. Se entrena un árbol de decisión sobre cada muestra.
3. En cada nodo se considera solamente un **subconjunto aleatorio de características** para buscar la mejor división.
4. Las predicciones de los árboles se agregan.

Diferencia respecto de Bagging

En un Random Forest, además de variar las muestras, se introduce aleatoriedad en la selección de características candidatas en cada nodo.

Esto reduce la correlación entre los árboles y favorece la reducción de varianza.

Random Forests en Scikit-Learn

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
    n_jobs=-1, random_state=42)

rnd_clf.fit(X_train, y_train)

y_pred = rnd_clf.predict(X_test)
```

Parámetros relevantes:

- `n_estimators`: número de árboles.
- `max_leaf_nodes`: máximo de hojas por árbol.
- `max_features`: número de características candidatas por división.
- `bootstrap`: controla el muestreo de observaciones.

Valor predeterminado

En clasificación, `max_features="sqrt"`: se consideran aproximadamente $\sqrt{n}$ características en cada nodo, donde n es el número total de características.

Extra-Trees: árboles extremadamente aleatorios

Los Random Forests introducen aleatoriedad al seleccionar las características candidatas.
Los **Extra-Trees** añaden otra fuente de aleatoriedad:

- Seleccionan un subconjunto aleatorio de características.
- Generan umbrales aleatorios para esas características.
- Eligen la mejor división entre los umbrales propuestos.

Consecuencias

- Mayor diversidad entre árboles.
- Posible aumento del sesgo.
- Posible reducción de varianza.
- Entrenamiento frecuentemente más rápido.

En Scikit-Learn se implementan mediante `ExtraTreesClassifier` y `ExtraTreesRegressor`.
A diferencia de Random Forest, en Extra-Trees `bootstrap=False` por defecto.

Importancia de las características

Los Random Forests estiman la importancia de cada característica mediante la **disminución promedio de impureza** (*Mean Decrease in Impurity*, MDI).

Reducción ponderada de impureza

Para un nodo t, la contribución de su división es

$$\Delta I(t) = \frac{N_t}{N}\left[I(t) - \frac{N_L}{N_t} I(t_L) - \frac{N_R}{N_t} I(t_R)\right] = \underbrace{\frac{N_t}{N}}_{\text{Peso del nodo}}\left[\underbrace{I(t)}_{\text{Antes de dividir}} - \underbrace{\left(\frac{N_L}{N_t} I(t_L) + \frac{N_R}{N_t} I(t_R)\right)}_{\text{Después de dividir}}\right].$$

- N: número total de muestras del árbol.
- N_t: número de muestras que llegan al nodo t.
- N_L, N_R: muestras en los nodos hijos izquierdo y derecho.
- $I(t)$: impureza del nodo t (Gini, entropía, etc.).
- $I(t_L)$, $I(t_R)$: impurezas de los nodos hijos.

Importancia de las características

La importancia de X_j se calcula sumando las contribuciones de los nodos que utilizan X_j, promediando sobre los árboles y normalizando para obtener

$$\boxed{\sum_{j=1}^{n} \text{Importance}(X_j) = 1.}$$

Importancia de Características: Iris

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

iris = load_iris(as_frame=True)

rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)

rnd_clf.fit(iris.data, iris.target)

for name, importance in zip(iris.data.columns,rnd_clf.
     feature_importances_):
   print(name, round(importance, 2))
```

Obteniendo:

Característica	Importancia
Sepal length	0.11
Sepal width	0.02
Petal length	0.44
Petal width	0.42

Las medidas de los pétalos son las más importantes para este clasificador.

Importancia de Características: Dígitos

```
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml

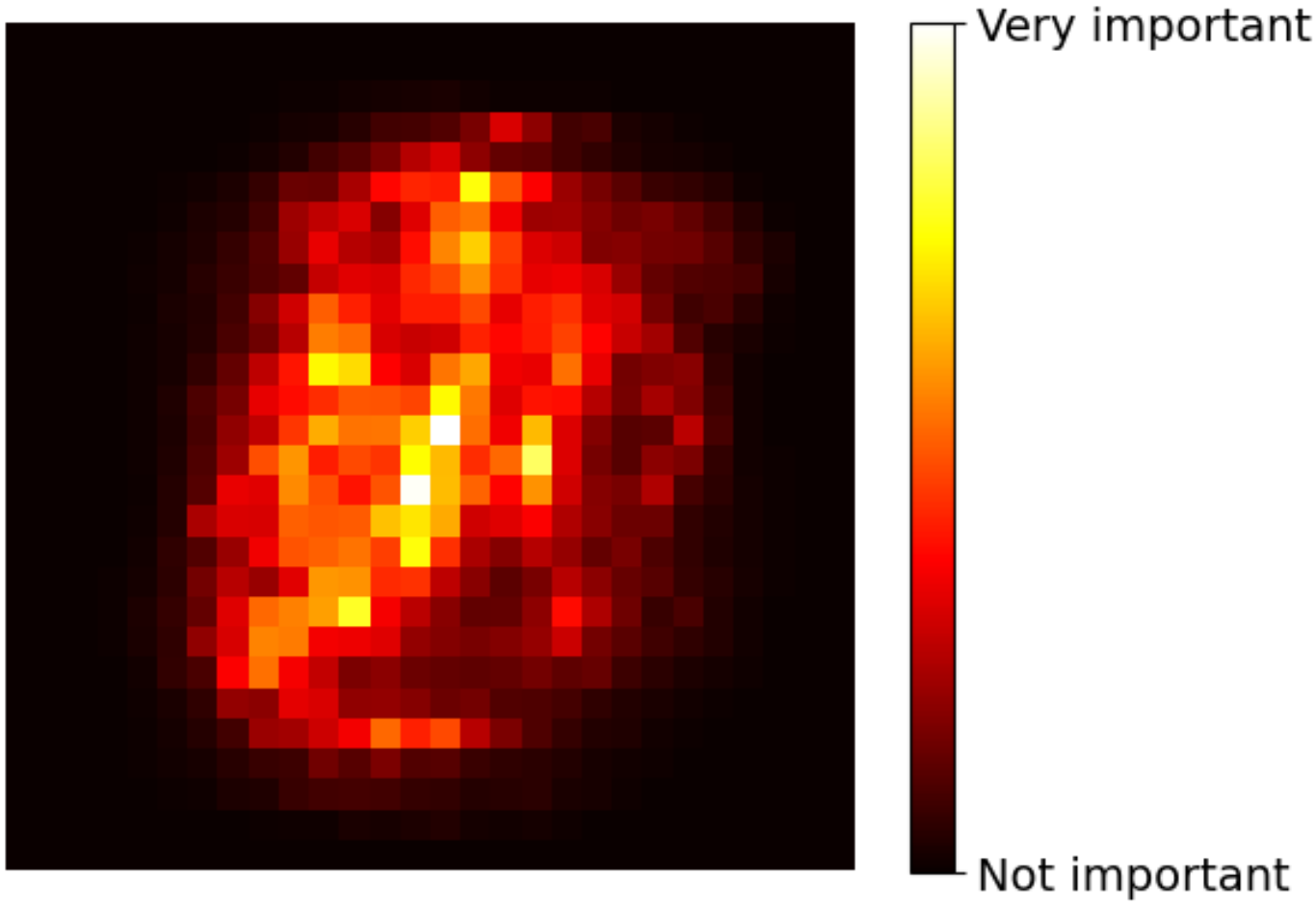
X_mnist, y_mnist = fetch_openml('mnist_784', return_X_y=True, as_frame=
    False, parser='auto')

rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rnd_clf.fit(X_mnist, y_mnist)

heatmap_image = rnd_clf.feature_importances_.reshape(28, 28)
plt.imshow(heatmap_image, cmap="hot")
cbar = plt.colorbar(ticks=[rnd_clf.feature_importances_.min(), rnd_clf.
    feature_importances_.max()])
cbar.ax.set_yticklabels(['Not important', 'Very important'], fontsize=14)
plt.axis("off")

plt.show()
```

Importancia de Características: Dígitos



Boosting: aprendizaje secuencial

Boosting combina múltiples predictores débiles para construir un predictor más fuerte.

A diferencia de bagging, los modelos se entrenan **secuencialmente**.

1. Entrenamos un primer predictor.
2. Identificamos los errores de sus predicciones.
3. Entrenamos otro predictor para corregirlos.
4. Repetimos el procedimiento.
5. Combinamos los predictores obtenidos.

Métodos principales

- **AdaBoost:** modifica los pesos de las observaciones.
- **Gradient Boosting:** ajusta nuevos predictores para reducir los errores del ensamble.

Desventaja

El entrenamiento secuencial limita la posibilidad de entrenar todos los modelos en paralelo.

Desventaja

No podemos entrenar en paralelo.

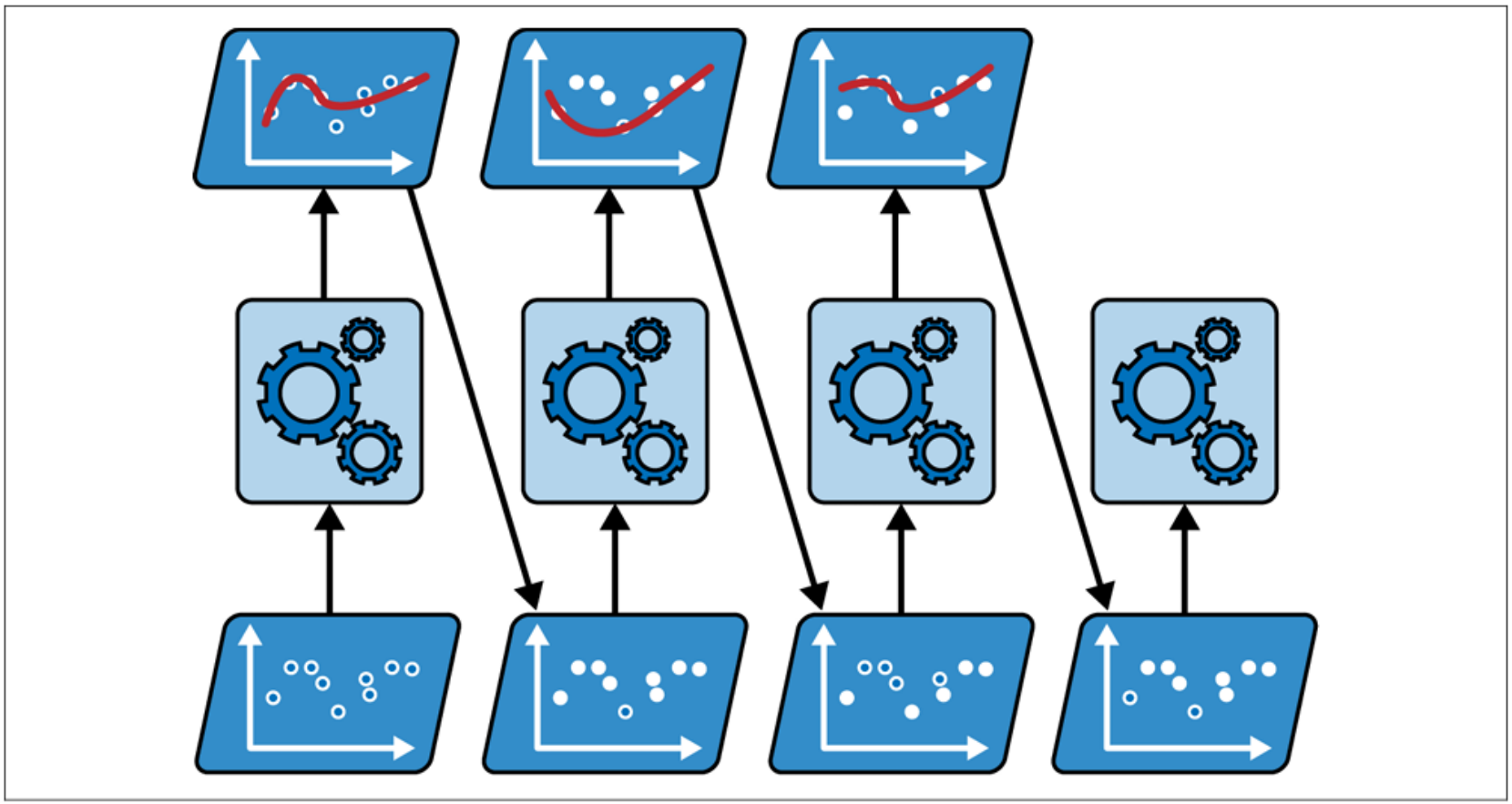


Figure 6-7. AdaBoost sequential training with instance weight updates

AdaBoost: principio de funcionamiento

AdaBoost (*Adaptive Boosting*) modifica la importancia de las observaciones durante el entrenamiento.

1. Inicialmente, todas las observaciones tienen el mismo peso:

$$w_i = \frac{1}{m}, \qquad i = 1, \ldots, m.$$

2. Entrenamos un clasificador utilizando los pesos.
3. Calculamos su error ponderado.
4. Aumentamos los pesos de las observaciones incorrectamente clasificadas.
5. Entrenamos el siguiente clasificador con los pesos actualizados.

Intuición

Cada nuevo clasificador presta mayor atención a los ejemplos que resultaron difíciles para los clasificadores anteriores.

AdaBoost: error y peso del predictor

Para el clasificador j, su error ponderado es

$$\boxed{r_j = \sum_{i=1}^{m} w_i \, \mathbb{I}\big(\hat{y}_j(x_i) \neq y_i\big)}$$

suponiendo que los pesos están normalizados: $\sum_i w_i = 1$.

Peso del clasificador

En el esquema binario:

$$\boxed{\alpha_j = \eta \log\left(\frac{1 - r_j}{r_j}\right),}$$

donde η es la tasa de aprendizaje.

- Si r_j es pequeño, α_j es grande.
- Si $r_j = 0{,}5$, entonces $\alpha_j = 0$.
- Si $r_j > 0{,}5$, el peso se vuelve negativo.

AdaBoost: actualización de pesos

Una vez calculado α_j, se actualizan los pesos de las observaciones[2]:

$$\widetilde{w}_i = \begin{cases} w_i, & \hat{y}_j(x_i) = y_i, \\ w_i e^{\alpha_j}, & \hat{y}_j(x_i) \neq y_i. \end{cases}$$

Después se normalizan:

$$w_i^{nuevo} = \frac{\widetilde{w}_i}{\sum_{\ell=1}^{m} \widetilde{w}_\ell}.$$

- El proceso se repite entrenando nuevos clasificadores con los pesos actualizados.
- En cada iteración se calcula el peso del clasificador y se ajustan los pesos de las observaciones.
- El algoritmo termina al alcanzar el número deseado de clasificadores o encontrar uno perfecto.

[2] si $\alpha < 0$ (o equivalentemente si $r_j > 0{,}5$) se descarta el modelo.

Predicción

Predicción

La clase final se obtiene mediante una votación ponderada:

$$\hat{y}(x) = \arg\max_k \sum_{j=1}^{M} \alpha_j \mathbb{I}\big(\hat{y}_j(x) = k\big).$$

Los clasificadores más precisos tienen mayor influencia en la predicción final.

AdaBoost en Scikit-Learn

Un predictor base común es un árbol de decisión de profundidad 1, llamado *decision stump*.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada_clf = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=30,
    learning_rate=0.5,
    random_state=42
)

ada_clf.fit(X_train, y_train)
```

- `n_estimators`: número máximo de clasificadores.
- `learning_rate`: controla la contribución de los clasificadores.
- `max_depth=1`: árboles simples, con una sola división.

Nota

Las versiones recientes de Scikit-Learn utilizan SAMME y ya no requieren el argumento `algorithm="SAMME"` mostrado en el libro.

AdaBoost: ¿cómo se utilizan los pesos al entrenar?

En AdaBoost, cada nuevo clasificador se entrena con las **mismas observaciones**, pero con pesos actualizados.

- **Árboles de decisión:** Los pesos se utilizan para calcular la impureza de los nodos y seleccionar las divisiones.

  Por ejemplo, el índice Gini ponderado es

$$G(t) = 1 - \sum_{k=1}^{K} \left(\frac{\sum_{i \in t,\, y_i = k} w_i}{\sum_{i \in t} w_i} \right)^2 .$$

  Las observaciones con mayor peso tienen más influencia en la elección de las divisiones.

- **Otros algoritmos:** Los pesos pueden incorporarse en la función de pérdida (p. ej., regresión logística y SVM), penalizando más los errores en observaciones con mayor peso.

Implementación en Scikit-Learn

```
modelo = DecisionTreeClassifier(max_depth=1)
modelo.fit(X_train, y_train, sample_weight=pesos)
```

Gradient Boosting

Gradient Boosting también construye un ensamble de forma secuencial.
Sin embargo, en lugar de modificar los pesos de las observaciones, ajusta nuevos modelos para corregir los errores del ensamble anterior.

Ejemplo con regresión y error cuadrático

Sea $F_1(x)$ el primer predictor.
Calculamos sus residuos:

$$r_i^{(1)} = y_i - F_1(x_i).$$

Entrenamos un segundo modelo h_2 para aproximar esos residuos.
El nuevo ensamble es

$$F_2(x) = F_1(x) + \eta h_2(x).$$

De manera general,

$$\boxed{F_M(x) = F_0(x) + \eta \sum_{j=1}^{M} h_j(x).}$$

Para otras funciones de pérdida, los modelos se ajustan a los gradientes negativos de la pérdida.

Gradient Boosting: primer árbol

Generamos un conjunto de datos de regresión:

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

m = 100
rng = np.random.default_rng(seed=42)

X = rng.random((m, 1)) - 0.5
noise = 0.05 * rng.standard_normal(m)

y = 3 * X[:, 0]**2 + noise

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

El primer árbol intenta aproximar directamente la función objetivo:

$$y = 3x^2 + \varepsilon,$$

donde ε representa ruido aleatorio.

Gradient Boosting: árboles sobre residuos

Entrenamos un segundo árbol sobre los residuos del primero:

```
y2 = y - tree_reg1.predict(X)

tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)
```

Después ajustamos un tercer árbol:

```
y3 = y2 - tree_reg2.predict(X)

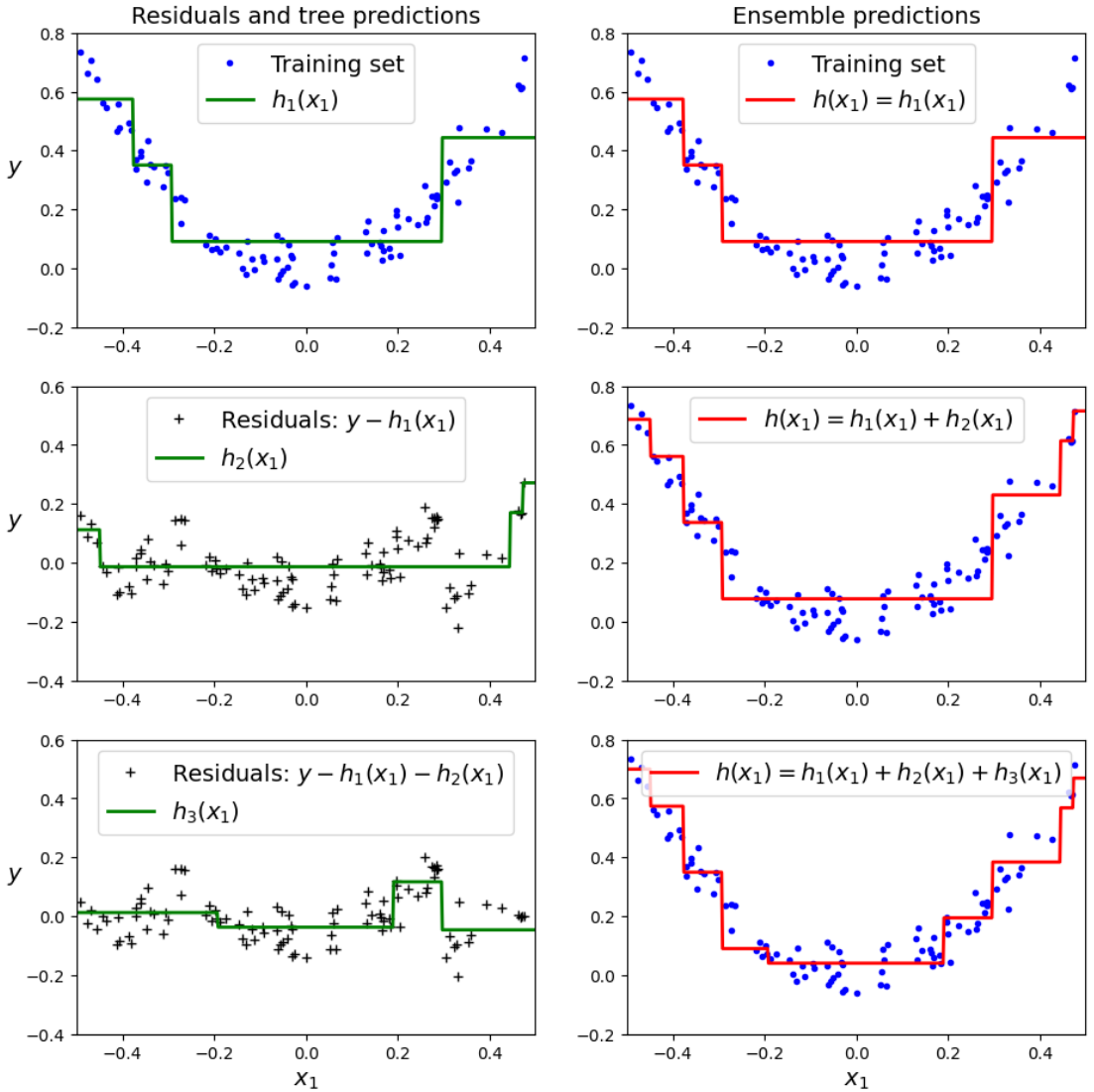
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
tree_reg3.fit(X, y3)
```

La predicción final se obtiene sumando:

$$F_3(x) = h_1(x) + h_2(x) + h_3(x).$$

Cada árbol intenta corregir lo que los anteriores todavía no lograron explicar.

Ejemplo anterior:



GradientBoostingRegressor

Scikit-Learn implementa directamente este procedimiento.

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(
    max_depth=2,
    n_estimators=3,
    learning_rate=1.0,
    random_state=42
)

gbrt.fit(X, y)
```

Hiperparámetros

- `max_depth`: complejidad de los árboles.
- `n_estimators`: número de etapas.
- `learning_rate`: magnitud de la contribución de cada árbol.

Gradient Boosting: Learning Rate

La tasa de aprendizaje controla cuánto contribuye cada nuevo árbol al ensamble:

$$\boxed{F_j(x) = F_{j-1}(x) + \eta h_j(x).}$$

Tasa de aprendizaje pequeña

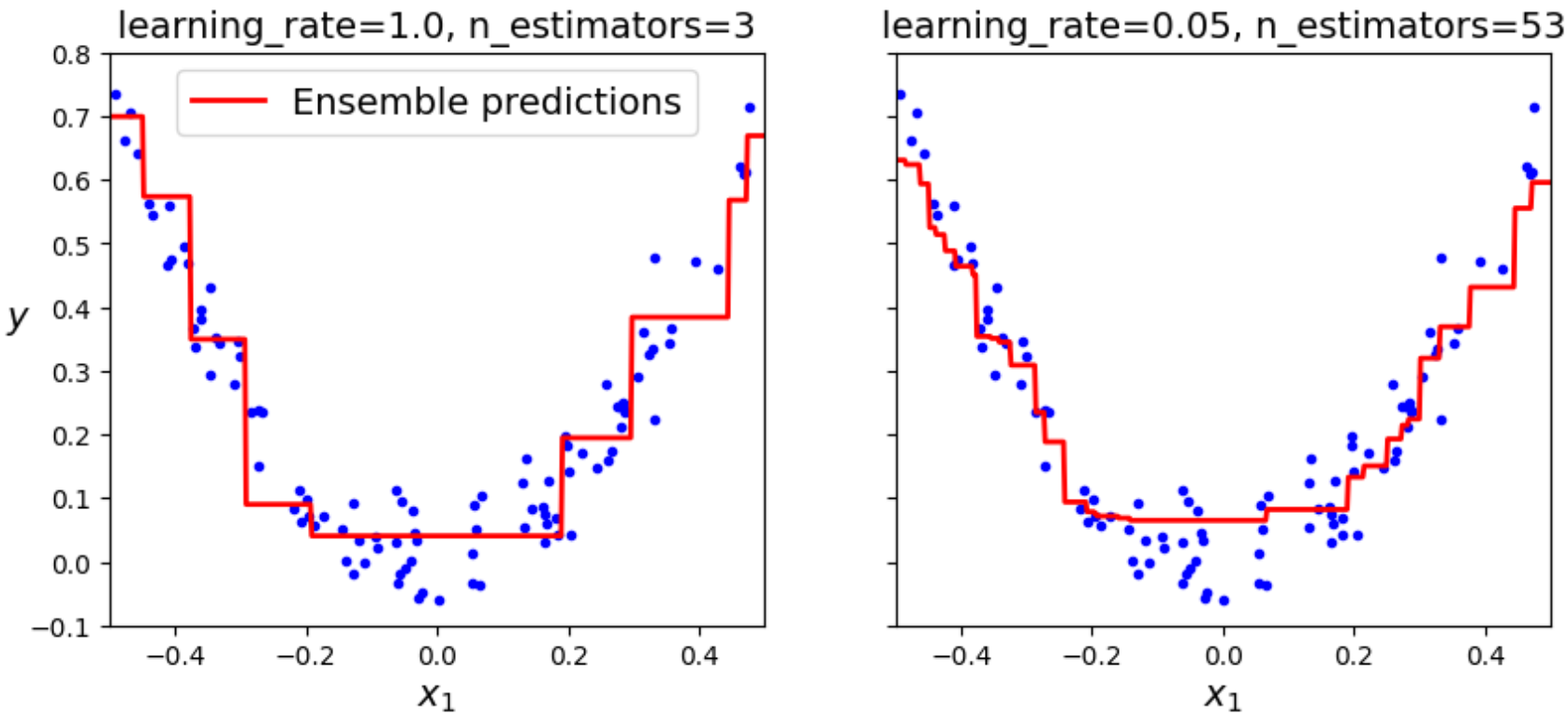
- Cada árbol introduce una corrección menor.
- Normalmente se necesitan más árboles.
- Puede mejorar la generalización.
- Aumenta el costo computacional.

Tasa de aprendizaje grande

- Las correcciones son más pronunciadas.
- El modelo puede ajustarse rápidamente.
- Puede aumentar el riesgo de sobreajuste.

Esta técnica de regularización se conoce como **shrinkage**.

Ejemplos


learning_rate=1.0, n_estimators=3
Ensemble predictions
y
x_1
learning_rate=0.05, n_estimators=53
x_1

Early Stopping en Gradient Boosting

Agregar demasiados árboles puede provocar sobreajuste.
Una estrategia es detener el entrenamiento cuando la métrica de validación deja de mejorar.

```
gbrt_best = GradientBoostingRegressor(
    max_depth=2,
    learning_rate=0.05,
    n_estimators=500,
    n_iter_no_change=10,
    random_state=42
)

gbrt_best.fit(X, y)
print(gbrt_best.n_estimators_)
```

En este ejemplo se entrenan efectivamente 53 etapas.

- `n_iter_no_change=10`: tolera diez iteraciones consecutivas sin mejora significativa.
- `validation_fraction`: fracción destinada a validación (default = 0.1).
- `tol`: umbral para considerar una mejora como significativa (default = 0.0001).

Stochastic Gradient Boosting

Podemos introducir aleatoriedad en Gradient Boosting entrenando cada árbol con una muestra aleatoria del conjunto de entrenamiento.
El hiperparámetro es `subsample`.

Ejemplo

Si establecemos

$$\texttt{subsample} = 0{,}25,$$

cada árbol utiliza aproximadamente el 25 % de las observaciones de entrenamiento.

Esto puede producir:

- Menor tiempo de entrenamiento por árbol.
- Mayor diversidad entre los árboles.
- Menor varianza.
- Un posible aumento del sesgo.

Compromiso

La fracción de muestreo debe elegirse de acuerdo con el tamaño y las características del conjunto de datos.

Gradient Boosting: ¿por qué funciona en regresión?

Supongamos que entrenamos un primer árbol $h_1(x)$ para aproximar los valores objetivo:

$$F_1(x) = h_1(x).$$

Como el árbol no es perfecto, calculamos los **residuos**:

$$\boxed{r_i^{(1)} = y_i - F_1(x_i).}$$

Luego, entrenamos un segundo árbol $h_2(x)$ utilizando los residuos como variable objetivo:

$$h_2(x_i) \approx y_i - F_1(x_i).$$

Al sumar ambos árboles, obtenemos:

$$\begin{aligned} F_2(x_i) &= F_1(x_i) + h_2(x_i) \\ &\approx F_1(x_i) + y_i - F_1(x_i) \\ &= y_i. \end{aligned}$$

Por tanto, el segundo árbol intenta **corregir los errores del primero**.

Gradient Boosting y descenso del gradiente

Consideremos la función de pérdida cuadrática para una observación:

$$L(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2.$$

Derivando respecto a la predicción $F(x_i)$:

$$\frac{\partial L}{\partial F(x_i)} = F(x_i) - y_i.$$

Por lo tanto, el **gradiente negativo** es

$$\boxed{-\frac{\partial L}{\partial F(x_i)} = y_i - F(x_i) = r_i.}$$

¡El gradiente negativo coincide con el residuo!

Entrenar un árbol sobre los residuos equivale a aproximar una dirección de descenso de la función de pérdida.

El ensamble se actualiza mediante

$$\boxed{F_j(x) = F_{j-1}(x) + \eta h_j(x),}$$

donde h_j aproxima los gradientes negativos y η controla el tamaño de la corrección.

Histogram-Based Gradient Boosting

próxima clase...

¡Gracias!